UGP Final Presentation · Parts I and II

Neural Networks for Fluid Flow

Part I: real airfoil wakes · Part II: pretraining

Aryamann Srivastava, 4th year Undergraduate, Dept. of AE

Supervisor: Prof. Rajesh Ranjan · November 2026

What is Covered

1	2	3	4	5
Part I: the problem	Part I: our approach	Part I: the result	Part II: the question	Part II: the findings
Forecasting a real wake	Model, scoring, bands	80.1 score	What pretraining buys	Five results, next steps

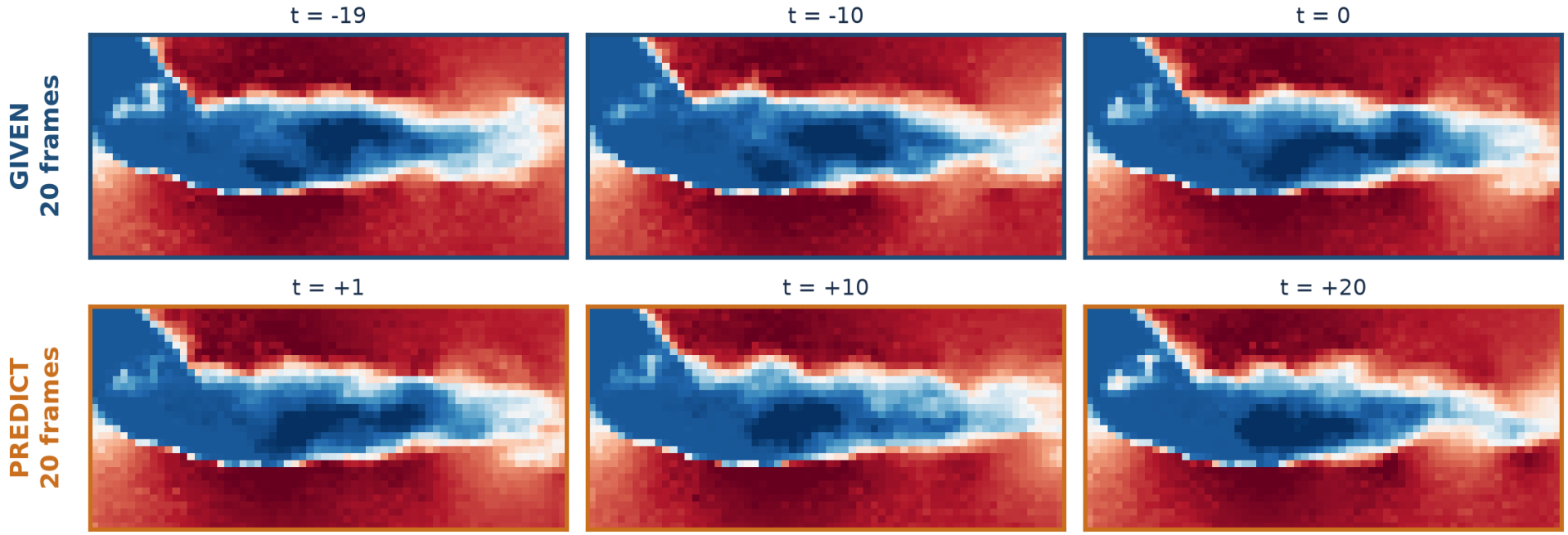
Part I

Forecasting Real Airfoil Wakes :
Neural Operators on PIV Measurements

NeurIPS 2026 RealPDE Competition · Track 1: Sim2Real

This Is the Data

Streamwise velocity, measured by PIV in a water tunnel



Given the top row → predict the bottom row.

The Task, in a Nutshell

One sentence, then the numbers

Given 20 measured frames, predict the next 20.

32 × 64	2	81	69,000
grid points per frame	velocity components u, v	training runs	real frames total

Conditions

Training Reynolds 3,750 – 26,700 ; angle of attack 0° – 20°

Testing unseen conditions, reaching Re 27,975 which happens to be above the training maximum

What We Actually Submit

A working program

Anatomy of a submission



The package must expose one function:

```
predict(input) → { prediction, lower, upper }
```

Graders call it in an isolated container: PyTorch and NumPy only, no internet, 5-minute limit.

A 250 MB zip. The graders import it, hand it 20 frames, and read back three arrays.

Why This Is Hard

Three independent difficulties

Turbulence

Small errors amplify.
20 frames is a long horizon.

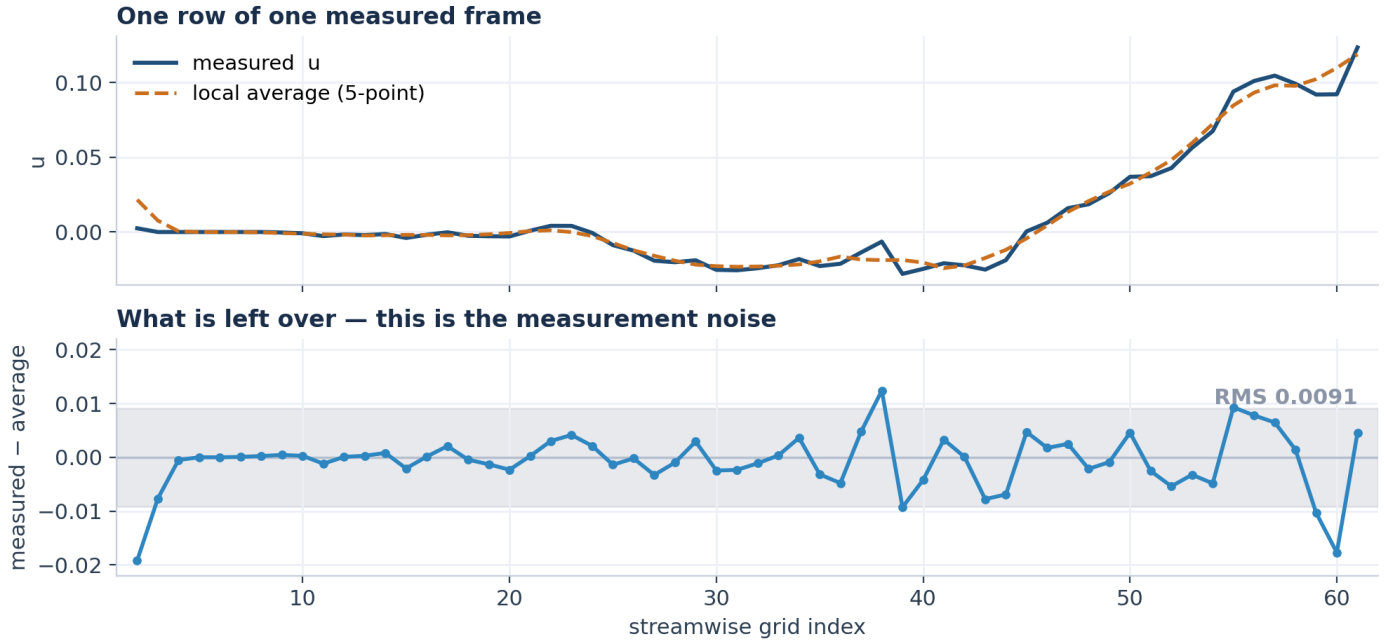
Extrapolation

Test conditions were
never seen in training.

Irreducible noise

The targets themselves
are noisy measurements.

The targets are noisy: 2.8% of the velocity range is measurement error, and no model can predict it



The Rules

Constraints to ensure before every decision

256 MB

total package size. The given model is 201 MB.

5 minutes

hard runtime limit, and speed is scored.

1 per day

submissions. One real measurement per 24 h.

Top 10 only

are shortlisted for the final private re-scoring.

One measurement per day makes guessing unaffordable.

Every submission had to be justified by a written prediction, and every prediction checked afterwards.

How a Submission Is Scored

Five channels to score on : the combining weights were never published

rel_l2	overall velocity error	**46.7%**
tke	turbulent kinetic energy	**10.0%**
mvpe	error at 9 wake probes	**9.4%**
time	how fast the code runs	**9.7%**
sps	quality of the UNCERTAINTY	**24.7%**

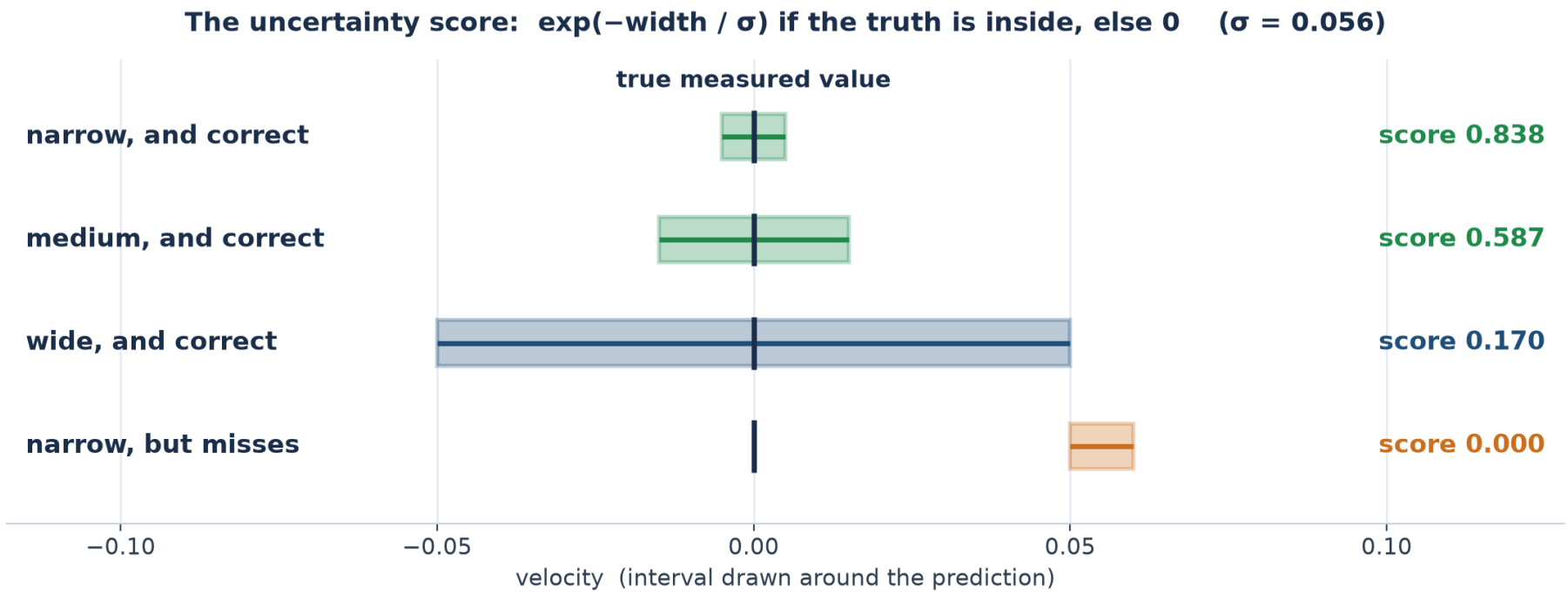
We recovered the hidden weights

Built a small tool that reads the public Codabench leaderboard, which let us solve for the weights of all metrics

This turned revealed that uncertainty is worth a quarter of the score.

The Unusual Channel: Uncertainty

We may return a lower and upper bound for every value



Narrow and correct is the only thing that pays. Being cautious is punished almost as hard as being wrong.

The Tool built

A small tool reading Codabench's public leaderboard endpoint

The problem

- The leaderboard shows one number per team.
- That number hides which of the five channels you are losing on.
- With one submission per day, spending them to find out is unaffordable.

What I built

- A small tool that reads Codabench's public API directly.
- Pulls every team's per-channel scores, not just the total.
- Refreshes the standings on demand during analysis.

What it bought us

- Showed uncertainty had a whopping 25% weightage.
- Concludes that SPS is the metric to be the most focussed on because it is the most volatile.
- Let us reverse-engineer the leader's model-vs-uncertainty split.

Is this allowed?

Yes. It reads the same public endpoint the website itself uses which we use for offline analysis on our local.

Approach 1 of 3: The Model

Improving the forecast itself

Fine-tuning	continue training the given simulation model on our 81 real runs
Model soup	train several times, then average the weights
Ensembling	run several models, average the answers, provide an offset if needed
TKE head	a second network that rescales the predicted fluctuations

Six weeks of this made the model better on unseen Reynolds numbers.

Approach 2 of 3: Uncertainty

Uncertainty is 24.7% of the score. Here is how we draw the bands

Three simple steps

1 Guess the error: a network estimates how wrong the forecast is at each point

2 Choose the width: expect a big error, draw a wide band; expect a small one, draw a narrow band

3 Centre it: shift the band to where it is most likely to catch the truth

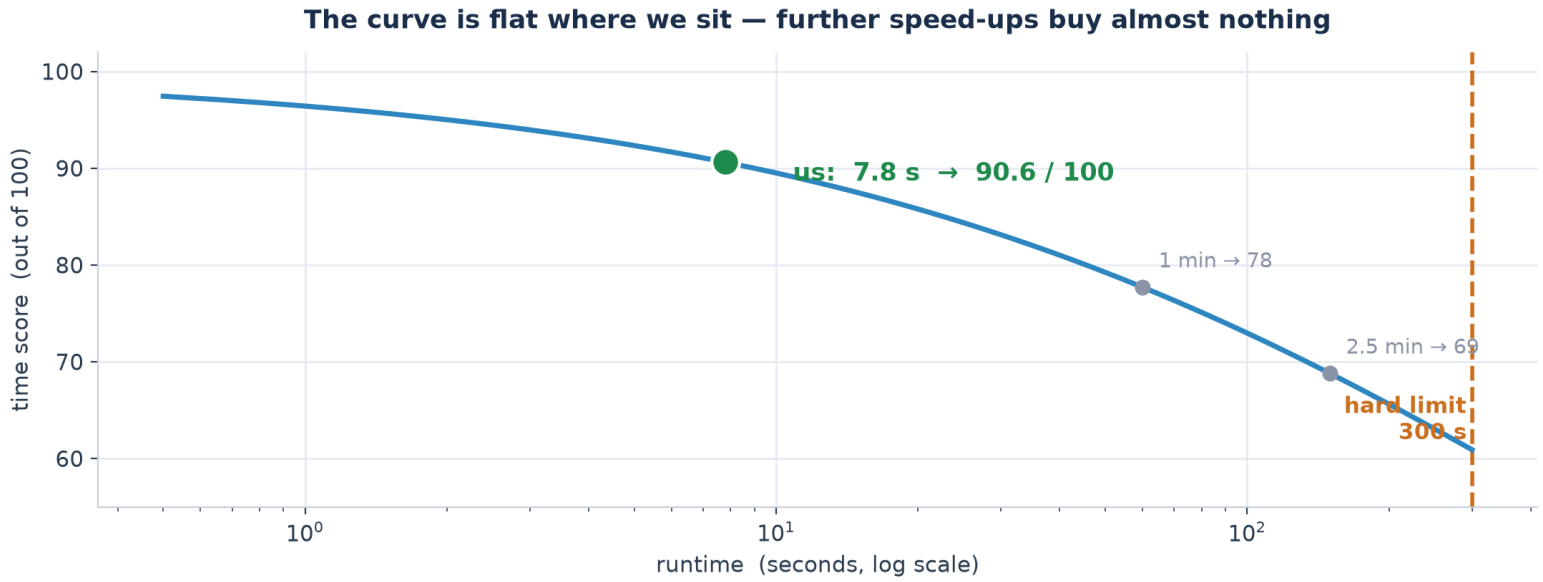
The forecast itself is never changed, so improving the bands cannot hurt accuracy.

In plain words

Like a weather forecast: “20°C, give or take 2”. We give the forecast and the “give or take” for every point. A small “give or take” that still turns out right earns the most points.

Approach 3 of 3: Speed

Why runtime is scored at all, and how much it was worth



What we did

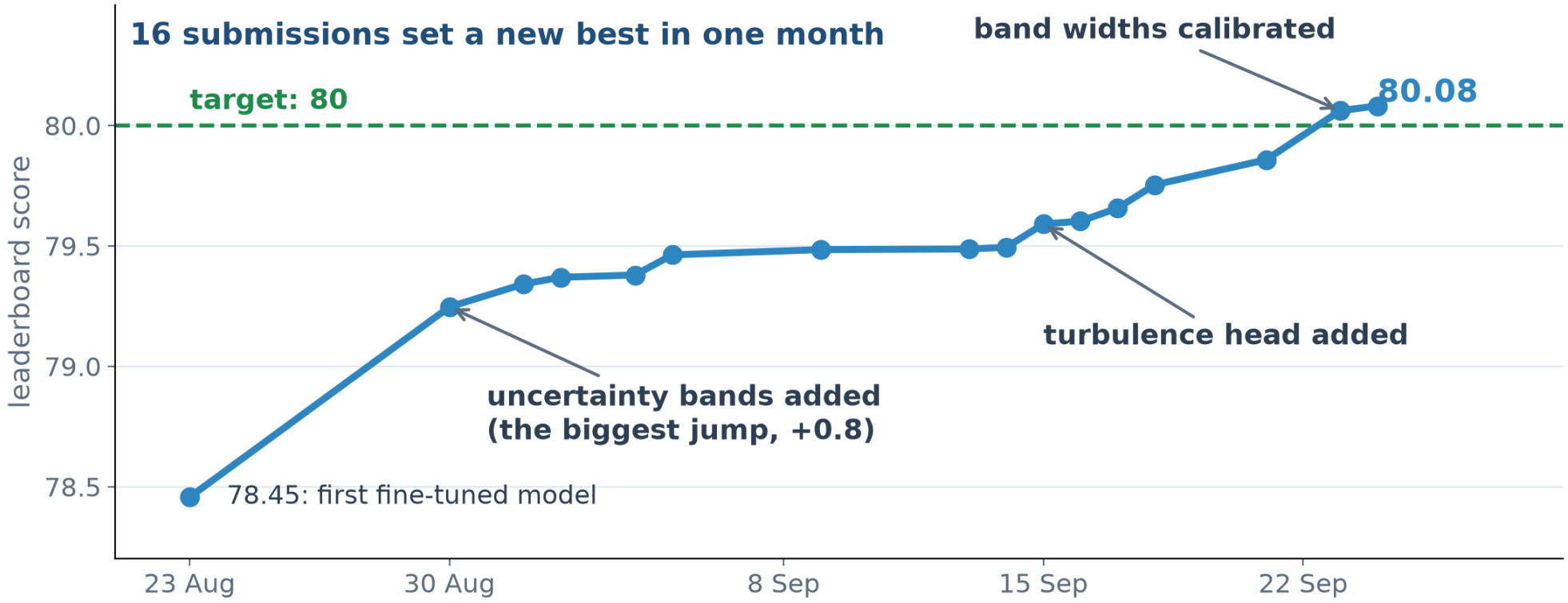
- Moved per-point work off the CPU onto the GPU.
- Merged three small networks into one.
- Stored weights at half precision.

Why we stopped

We are already on the flat part of the curve. Going from 7.8 s to 4 s would gain only +0.25 of a point and the one change big enough to matter destroyed accuracy.

Score Progression

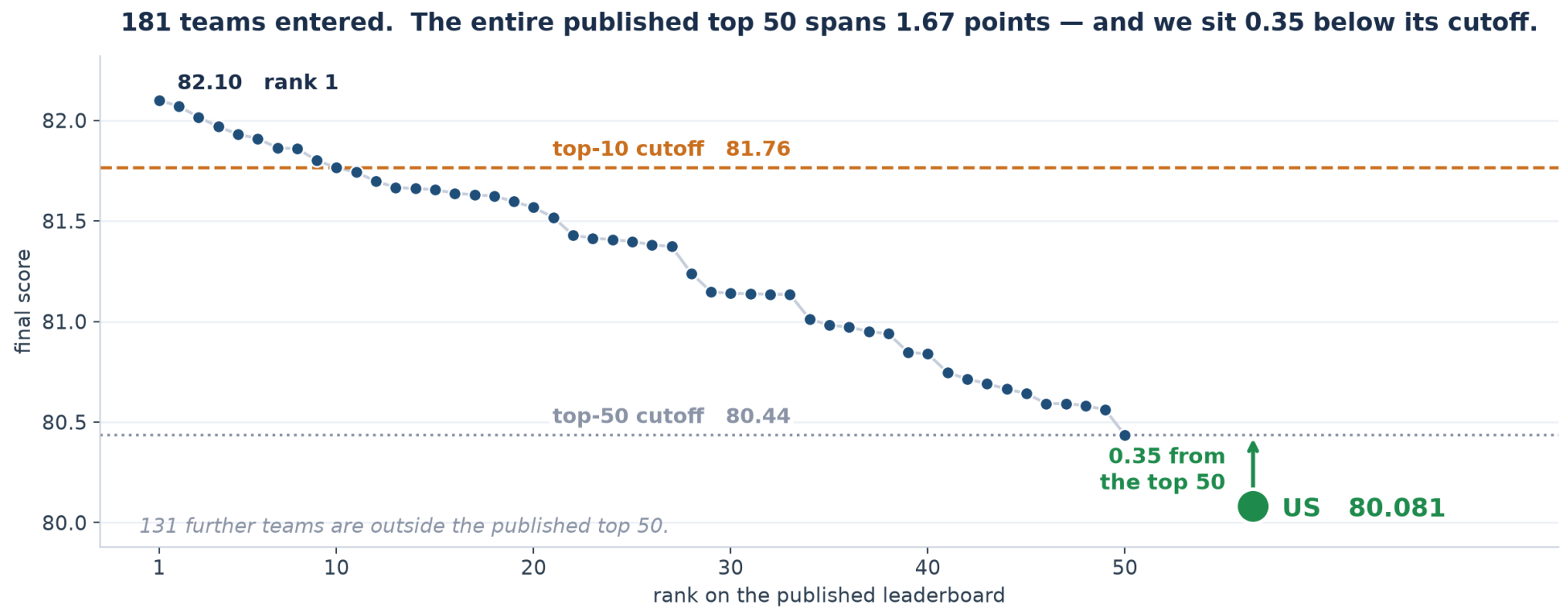
Every submission that set a new best, on the day it was scored (23 Aug to 24 Sep)



16 submissions set a new best, from 78.45 to 80.08. The last two steps changed only the bands.

Where We Stand

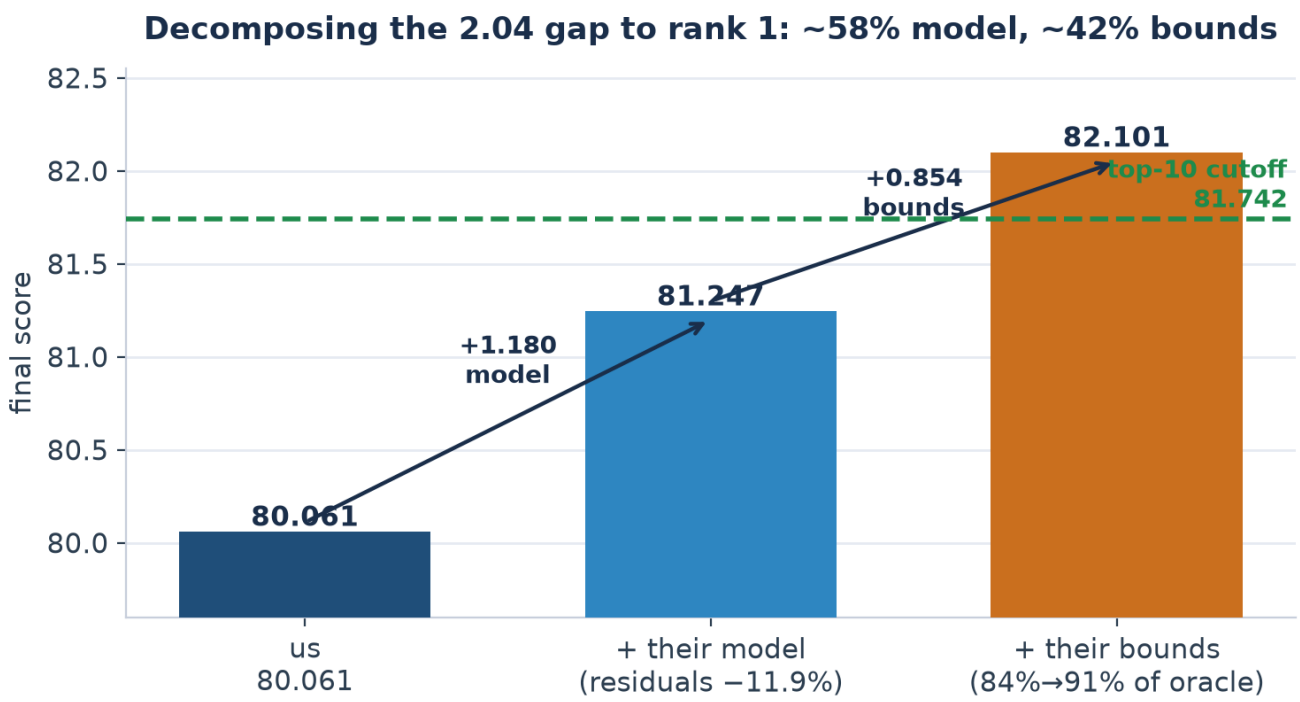
Development leaderboard on 24 September 2026: 181 teams



A very tight field: 2.5% separates rank 1 from outside the top 50.

What the Gap Consists Of

Reverse-engineered from the rank-1 competitor's published scores



How we know

- Their errors are 10–16% smaller and they are 30% FASTER.
- Faster AND more accurate cannot come from tuning.
- Against a perfect-uncertainty : we are at 83.6%, they are at 90.7%.
- So: ~58% better model, ~42% better uncertainty.

Part I: Where We Landed

Final development leaderboard, about 182 teams

80.081	72.65	81.901	82.218
our score	organisers' baseline	top-10 cutoff	development leader

The ceiling, measured

Even a mathematically PERFECT uncertainty model scores 81.727, below the top-10 cutoff (81.765 on 24 Sep).

So a top-10 place provably requires a better underlying model, not better bounds, and the model is converged.

What it led to

- Finished just outside the top 50: the whole top 50 sat within 1.6 points.
- The forecast, not the bands, was the limit.
- Open question: would pretraining on related data have helped?
- Part II answers it on a flow we control.

Part II

What Does Pretraining Really Buy?

Rebuilding Aurora, a large weather AI model, from scratch in JAX, and testing what makes it work on swirling fluid flow

Where We Started

UGP-I ended with a question it could not test

UGP-I: RealPDE competition

NeurIPS 2026, sim-to-real track

80.08

development-phase score

Task: forecast real, measured airflow past a wing, from only 81 lab recordings

The question it left open

Would a model pretrained on lots of related data have done better?

- Outside pretrained models were banned
- Too little real data to try it
- So UGP-II builds a setting where it can be tested properly

The Big Idea: Pretrain, Then Fine-tune

How today's large scientific AI models are trained

1 Pretrain

Learn from lots of related data

2 Fine-tune

Brief practice on the actual task

3 Predict

Use it on new cases

Pretrained

Like a student who studies every chapter, then practises the exam chapter

From scratch

Studies only the exam chapter

Aurora Says It Works, But Not Why

Microsoft's weather foundation model

25%

better than training from scratch

1.3 billion parameters, pretrained on over a million hours of Earth-system data

But which ingredient of the data creates that gain?

Amount

How much data

Variety

How many kinds of flow

Format

Mixed grids and variables

Closeness

How similar to the task

Aurora changes all four at once, and one rerun would need a lot of GPUs

Our Goal

Rebuild Aurora from scratch in JAX, then use it to find out which property of the pretraining data creates the benefit

1

Rebuild

Aurora from scratch in JAX, at small scale

2

Create

our own data solver: a fluid simulator

3

Test

each ingredient, one at a time

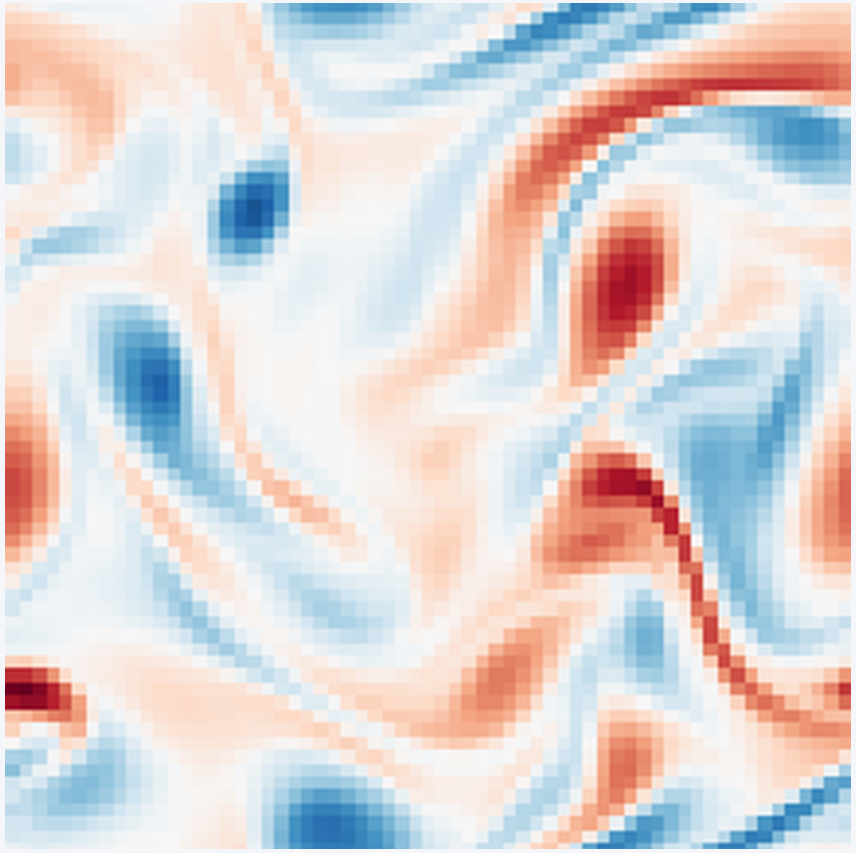
4

Check

the answer on real lab data

Why Kolmogorov Flow?

A classic test case: fluid in a box, kept moving by a steady push



Spin of the fluid: red one way, blue the other

1. **We create the data**

   Any amount, any setting, at no cost

2. **Two simple knobs**

   Change how the flow behaves, one at a time

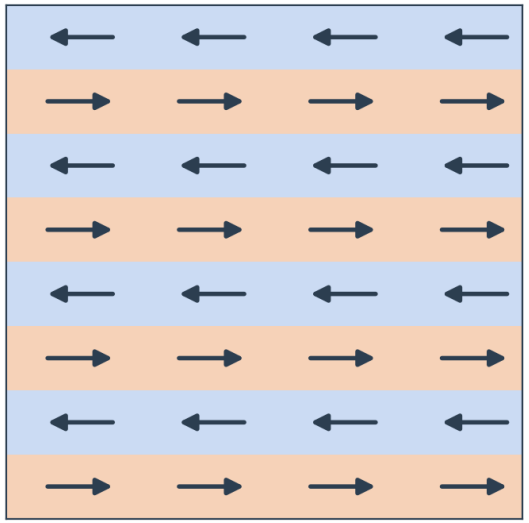
3. **Real turbulence**

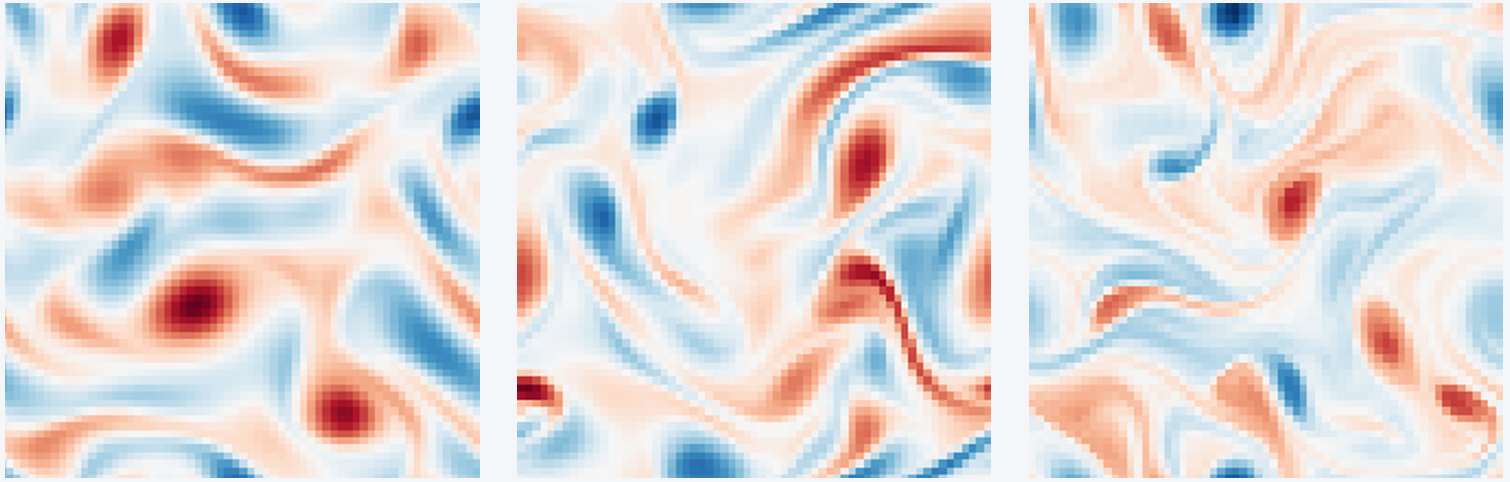
   Chaotic, so forecasting it is a genuine challenge

4. **Exact answers**

   Every prediction is scored against the true flow

Two Knobs Define Each Flow





Stirring pattern (kf)

Bands of fluid pushed left and right; kf = number of band pairs. Target: 4

Reynolds number (Re)

How wild the flow gets: thick like honey at low Re, thin like water at high Re. Target: 300

The Task: Predict the Next Moment

Two snapshots in, one snapshot out



a moment ago | now | next moment

Illustrative snapshots

Longer forecasts

Feed each prediction back in as the new input, step after step. We test 20 steps ahead.

How We Measure Success

Every prediction is compared with the true simulated flow

Error

How far the prediction is from the truth

0% = perfect

Beat the lazy guess

Must do better than "nothing changes"

the minimum bar

Long-forecast drift

Average error over 20 steps of feeding predictions back

stability

Pretraining gain

How much lower the error is than training from scratch

the key number

What We Built

Four verified pieces, each tested before use

Fluid simulator

10^{-9}

energy balance error: physically exact

Dataset

326

simulations across 33 flow settings

Aurora copy

5.1 M

parameters; matches Aurora's own code to 10^{-7}

Experiment system

1 → 1

one settings file gives one run and one result row

Inside the Model

Same building blocks as Aurora, written from scratch in JAX

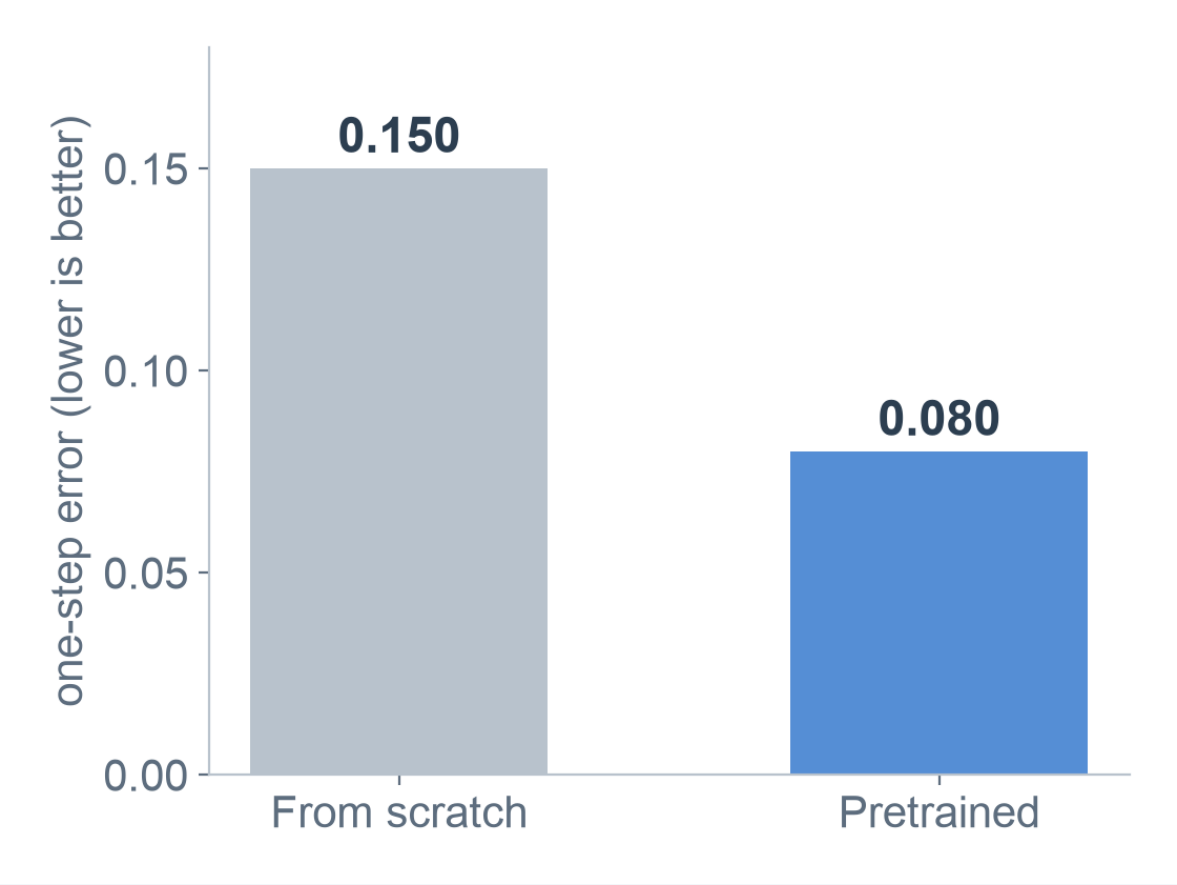
Two snapshots	Cut into patches	Patches share information	Zoom out and back in	Next snapshot
the flow now and a moment ago	256 small squares	attention, inside local windows	see big and small swirls	the forecast

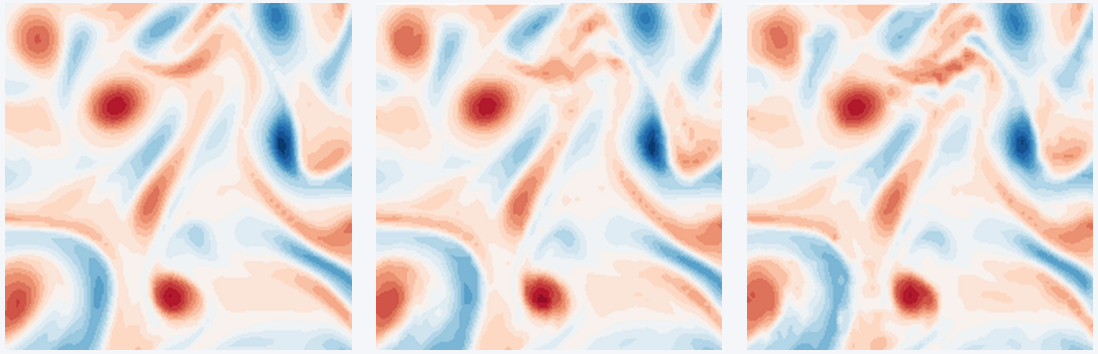
Attention: each patch learns which other patches matter for predicting it

Zooming: coarse views capture large swirls, fine views keep the detail

Result 1: Pretraining Halves the Error

42–50% lower one-step error, in every setup tested





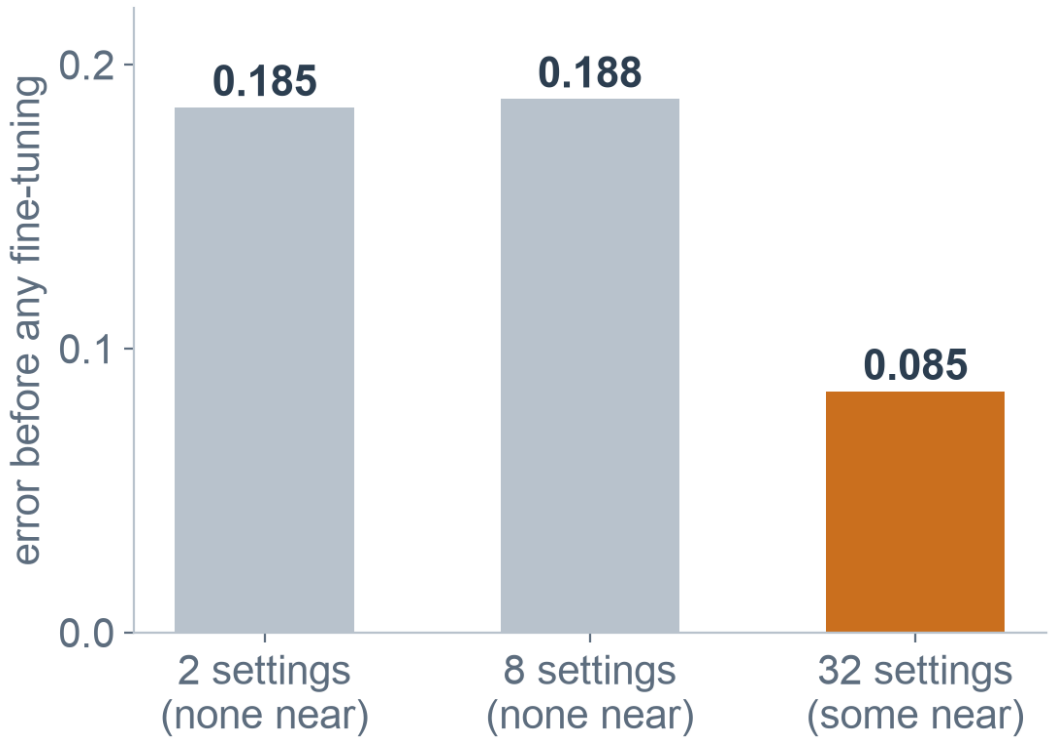
Truth | Pretrained | From scratch

Colours show the fluid's spin: red one way, blue the other, white still. One step ahead on a flow never seen in training: the pretrained forecast is closer to the truth (error 0.19 vs 0.28).

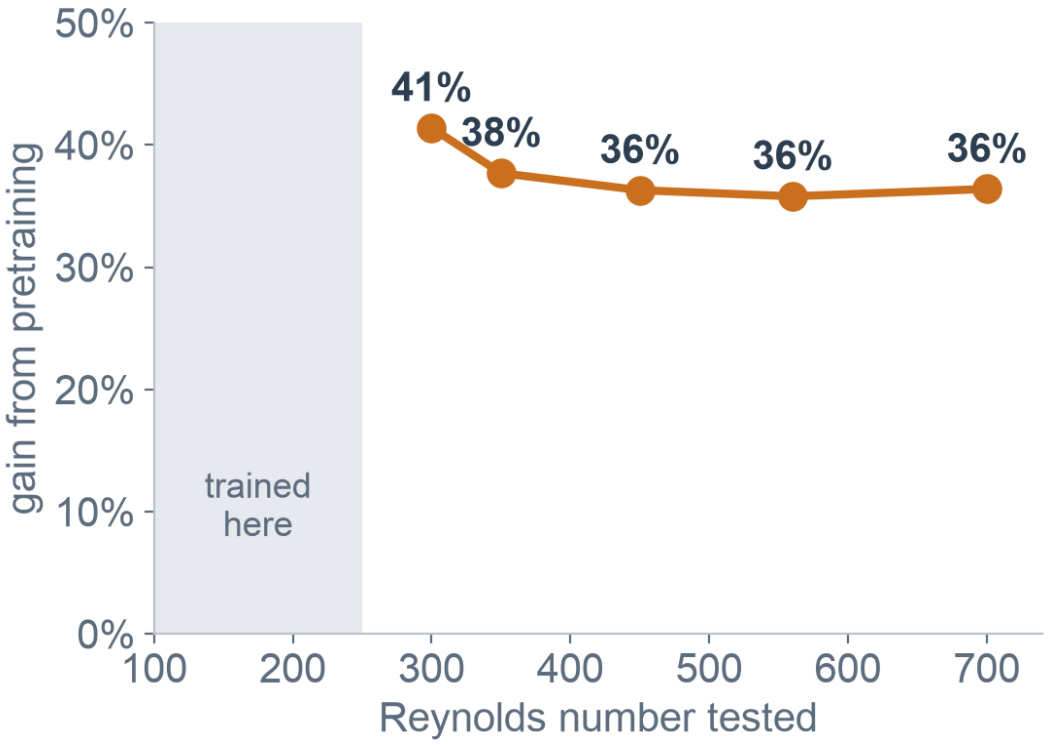
Result 2: Closeness Matters Most

More data or more variety added nothing; being near the task did

Data near the task gives a far better start

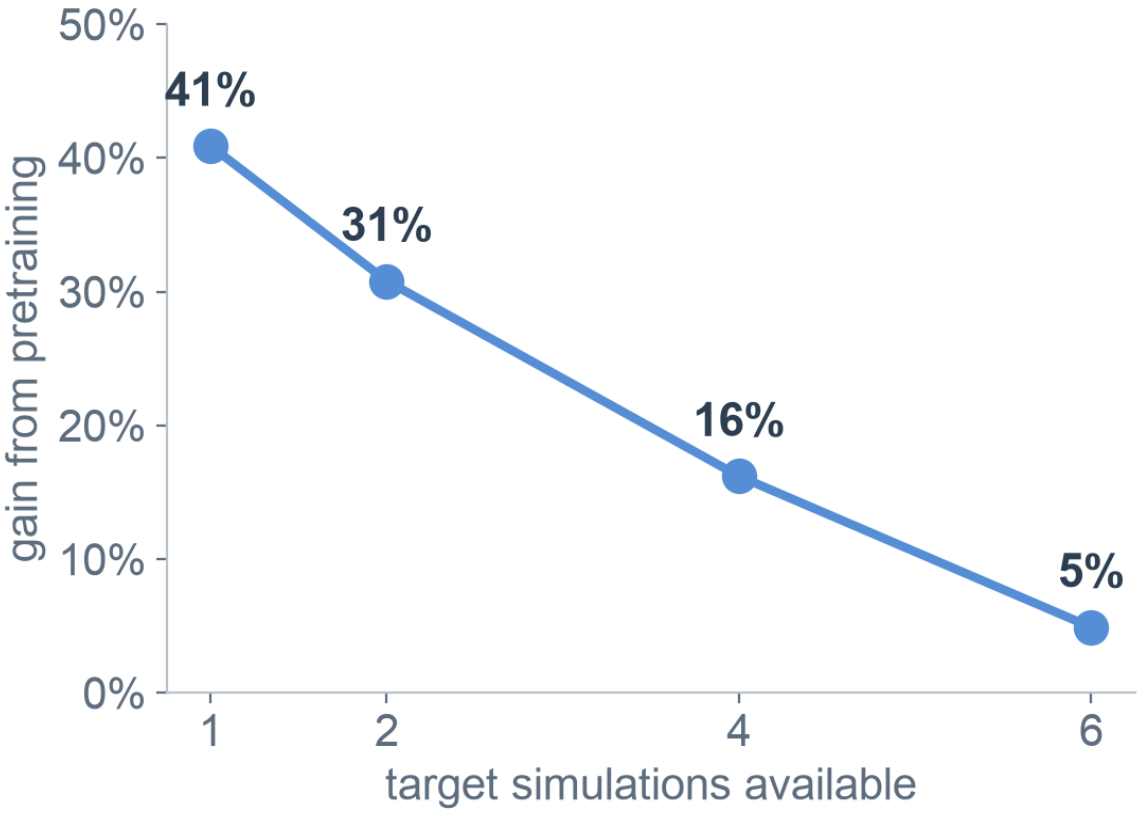


The gain holds far beyond the training range



Result 3: Most Valuable When Data Is Scarce

Pretraining pays off most when the task has little data of its own



41%

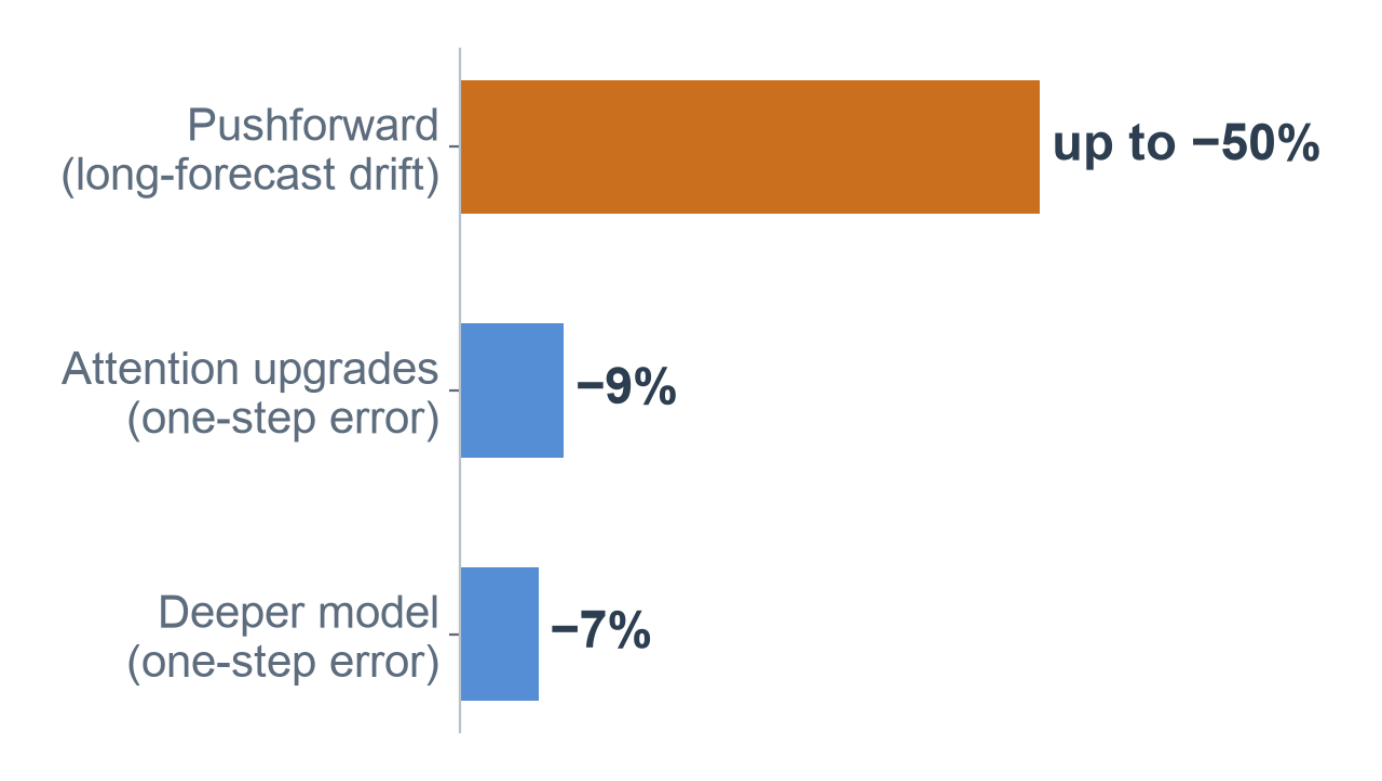
gain with a single target simulation

Ready out of the box

Even without fine-tuning, the pretrained model is within 6% of the best fine-tuned result

Result 4: What Makes the Model Better

Measured improvements from individual design choices



Pushforward

Train on the model's own predictions, so long forecasts stay steady

Attention upgrades

Two small changes, about the gain of doubling the depth at a fraction of the cost

Cheap fine-tuning (LoRA)

Corrects systematic errors: up to 71% where the base model is off

Result 5: Tested on Real Lab Data

Back to UGP-I: real, measured airflow past a wing (scores out of 100, higher is better)

UGP-I: competition development set

Model	Accuracy	Fluctuations	Mean flow	Overall
Organisers' FNO (kit baseline)	94.2	74.0	92.8	72.7
Our UGP-I entry (fine-tuned FNO + extras)	94.0	**78.6**	93.2	**80.1**

UGP-II: our held-out real recordings

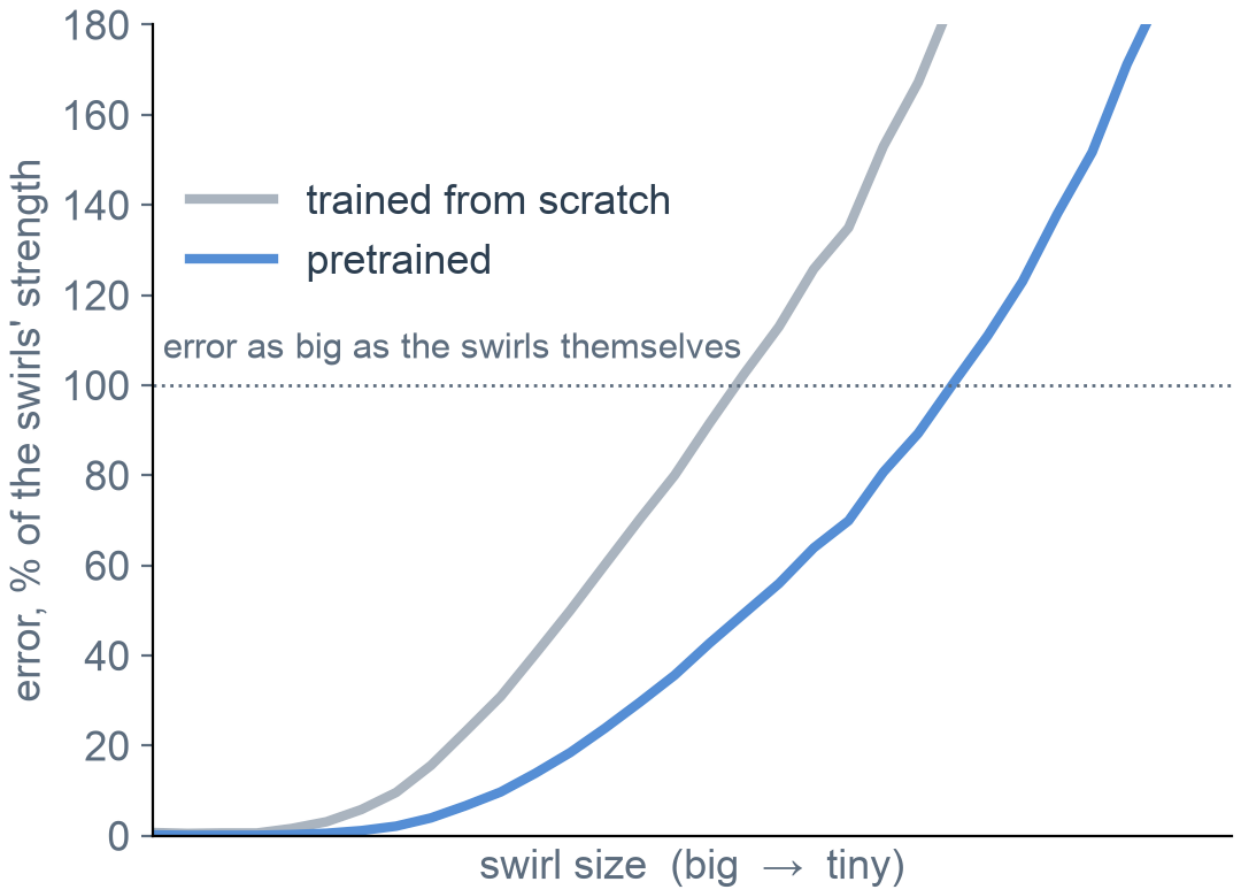
Model	Accuracy	Fluctuations	Mean flow	Overall
Organisers' FNO (fine-tuned on real data)	95.5	75.9	96.1	n/a
Our Aurora rebuild, real data only	**94.7**	75.6	94.2	n/a
Our Aurora rebuild, simulation-pretrained	**94.8**	76.1	94.4	n/a

Our rebuilt Aurora, seeing only 2 past frames (FNO: 20), comes within 1 point of the organisers' FNO, and simulation pretraining adds under 0.1: on real data, the real data is what counts.

The two test sets differ, so compare within each table. "Overall" also includes speed and uncertainty scores, which UGP-II did not measure.

What Still Limits Accuracy

We examined the 30 trained models to see where the remaining error lives



Tiny swirls

Big swirls are almost perfect (error under 1%); the error lives in the smallest swirls

The spin field

About 70% of all error is in the spin (vorticity), the field richest in tiny swirls

Data, not training time

Trained from scratch, the model memorises its 2 examples (32× better on them); pretrained models do not (1.2×)

Key Takeaways

1. Pretraining halves next-step forecast error
2. **Closeness to the task matters more than amount or variety**
3. The biggest payoff comes when task data is scarce
4. Training on its own predictions keeps long forecasts steady
5. Building the physics in keeps 20-step forecasts 13% steadier

Future Work

Where the current project can be taken further

1. **Physics built in**

   Tested: predicting only the spin and deriving the rest cut 20-step error by 13%; next, add pushforward

2. **Steadier long forecasts**

   Build on pushforward so predictions stay accurate many steps ahead

3. **Richer input history**

   Let the model know the order of its input frames, and see more of the past

4. **Scale and real data**

   A larger model and dataset, and better transfer from simulation to real measurements

Both Parts Together

What UGP-I and UGP-II teach as one project

1. Know what you are measured on: hidden score weights (I), next step vs long forecasts (II)
2. **Pretraining helps when it is close to the task and target data are scarce**
3. Far-away simulations did not help on the real wake (I and II)
4. Physics helps only when it is true for the data: hurt on camera data, helped on simulation
5. Write the prediction down first: every surprise becomes a result

Appendix

The details are in the final report, not on the slides

A. Part I: the scoring formulas, the uncertainty bands, the component ablation and the directions we closed

B. Part II: the solver, dataset, model and training details

C. All eleven Part II experiments and the full prediction record

D. Deriving velocity and pressure from the spin, and a glossary

Final report: UGP_Final_Report.pdf

Part I code and record: github.com/ARYAMANN7279/RealPDE-Track1

Part II code and results: github.com/ARYAMANN7279/aurora-from-scratch-jax

UGP · Parts I and II

Thank You

Part I: github.com/ARYAMANN7279/RealPDE-Track1

Part II: github.com/ARYAMANN7279/aurora-from-scratch-jax